

Sécurité des sessions & travail backend

Comment les sessions admin / marchand / équipe sont séparées, ce qui a été construit aujourd'hui (suppression de compte marchand, réinitialisation de mot de passe), et comment continuer à faire évoluer le projet avec Claude Code.

Projet : next-one (Power Delivery)

Date : 25 juillet 2026

Préparé pour : Basma Boutaib

Sommaire

1.	1. Comment les sessions sont séparées (admin / marchand / équipe)
2.	2. Où sont les failles connues et pourquoi elles existent
3.	3. Ce qui a été construit aujourd'hui — vue d'ensemble
4.	4. Détail fichier par fichier — suppression d'un marchand
5.	5. Détail fichier par fichier — réinitialisation de mot de passe
6.	6. Mesures de sécurité mises en place sur le reset de mot de passe
7.	7. Comment lire et comprendre ce code toi-même
8.	8. Comment tester ce qui a été livré
9.	9. Comment avancer sur les prochains prompts

1. Comment les sessions sont séparées (admin / marchand / équipe)

Il n'existe **pas** un système de session différent par type de compte. Il y a un seul mécanisme de session, et c'est le contenu de cette session qui détermine ce que chaque utilisateur peut voir ou faire.

1.1 Un seul cookie, un seul jeton

À la connexion ([app/api/auth/login/route.ts](#)), l'utilisateur reçoit un cookie httpOnly nommé `pd_session`. Ce cookie contient un jeton **JWT** (signé, pas chiffré — donc lisible mais pas falsifiable sans la clé secrète `AUTH_SECRET`) avec exactement deux informations :

```
{
  "sub": "id-de-l-utilisateur",
  "role": "admin" | "marchand" | "ramasseur" | "livreur" | "finance" | "sav" | "agent_confirmation"
}
```

Tout le cloisonnement entre "espace admin", "espace marchand" et "espace équipe terrain" repose sur ce champ `role`. C'est [lib/auth.ts](#) qui signe (`signSession`) et vérifie (`verifySessionToken`) ce jeton avec la librairie `jose`.

1.2 Deux couches de contrôle

Couche 1 — `proxy.ts` (l'équivalent de `middleware.ts` dans cette version de Next.js, renommé `proxy.ts`). Il s'exécute avant *toutes* les requêtes vers `/api/**` :

- Si l'URL est dans `PUBLIC_API_PATHS` (login, inscription, mot de passe oublié) → laisse passer sans vérifier de session.
- Sinon, lit le cookie `pd_session`, vérifie sa signature. Absent ou invalide → réponse `401` immédiate, la requête n'atteint jamais la route.
- Si valide, pose deux en-têtes internes (`x-pd-user-id`, `x-pd-user-role`) que les routes API liront ensuite — pas besoin de revérifier la signature JWT à chaque endpoint.

Important : à ce stade, on sait seulement "authenticifié ou pas" — pas encore "autorisé pour ce rôle".

Couche 2 — lib/api-utils.ts → fonction `requireUser(rolesAutorises)`, appelée en première ligne de chaque route API sensible :

```
const session = await requireUser(['admin']); // uniquement l'admin
const session = await requireUser(['marchand']); // uniquement un marchand
const session = await requireUser(); // n'importe quel rôle connecté
```

Si le rôle ne correspond pas → **403 Accès refusé pour ce rôle**. C'est ici, route par route, que se joue la vraie séparation "marchand ne peut pas toucher aux routes admin".

1.3 Et pour les pages (pas seulement l'API) ?

Le `proxy.ts` ne protège que `/api/**`. Les pages elles-mêmes (dans `app/admin/**`, `app/marchand/**`) doivent donc vérifier la session **elles-mêmes**, via `getPageSession()` (toujours dans `lib/auth.ts`), qui relit directement le cookie côté serveur.

Fichier	Vérifie la session avant d'afficher la page ?
<code>app/admin/layout.tsx</code>	✓ Oui — <code>redirect('/login')</code> si pas admin
<code>app/marchand/layout.tsx</code>	✗ Non — composant client, aucune redirection
<code>app/ramasseur/page.tsx</code>	✗ Non — composant client, aucune redirection

Concrètement : si quelqu'un de non connecté ouvre `/marchand` ou `/ramasseur` directement, il verra la coquille de la page (menu, en-tête) mais aucune donnée, puisque tous les appels vers `/api/**` échoueront avec **401**. Les données ne fuient pas, mais l'expérience est incohérente par rapport à l'admin. C'est documenté à la section suivante comme point à corriger si tu le souhaites.

2. Où sont les failles connues et pourquoi elles existent

Faible 1 — Pages marchand/ramasseur non gardées côté serveur.

Comme vu ci-dessus, `app/marchand/layout.tsx` et `app/ramasseur/page.tsx` n'ont pas l'équivalent du `redirect('/login')` que fait `app/admin/layout.tsx`. Aucune donnée ne fuit (l'API bloque quand même), mais ce n'est pas cohérent. *Correction proposée mais pas encore appliquée — dis-le si tu veux que je le fasse.*

Ce qui, en revanche, fonctionne correctement et a été vérifié en le testant réellement (pas en lisant le code) :

- Impossible de créer deux comptes marchand avec le même email (testé avec deux requêtes réelles).
- Impossible pour un marchand connecté d'appeler une route admin (`requireUser` renvoie `403`).
- Impossible d'accéder à `/api/**` sans cookie de session valide (`401`).

3. Ce qui a été construit aujourd'hui — vue d'ensemble

Deux demandes concrètes ont été traitées :

1. **Suppression d'un compte marchand** depuis l'espace admin (approuver/suspendre existaient déjà, supprimer n'existait pas).
2. **Réinitialisation de mot de passe en cas d'oubli**, sous deux formes : une déclenchée par l'admin (utilisable immédiatement), une en libre-service par email (nécessite une configuration SMTP que tu dois encore fournir).

Fichier	Statut	Rôle
<code>prisma/schema.prisma</code>	modifié	+2 colonnes sur <code>Utilisateur</code> pour le reset de mot de passe
<code>lib/auth.ts</code>	modifié	+3 fonctions : génération/hash de token, mot de passe temporaire
<code>lib/mailer.ts</code>	nouveau	Envoi d'email via SMTP (nodemailer)
<code>lib/api-client.ts</code>	modifié	+ fonction <code>apiDelete</code>
<code>lib/types.ts</code>	modifié	+ champ <code>utilisateurId</code> sur le type <code>Marchand</code>
<code>proxy.ts</code>	modifié	+2 routes publiques (mot de passe oublié / réinitialisation)
<code>app/api/marchands/[id]/route.ts</code>	nouveau	Suppression d'un marchand (DELETE)
<code>app/api/auth/mot-de-passe-oublie/route.ts</code>	nouveau	Demande de reset par email (public)
<code>app/api/auth/reinitialiser-mot-de-passe/route.ts</code>	nouveau	Valide le lien + change le mot de passe (public)
<code>app/api/utilisateurs/[id]/reinitialiser-mot-de-passe/route.ts</code>	nouveau	Reset déclenché par l'admin (protégé)
<code>app/admin/marchands/page.tsx</code>	modifié	Boutons "Supprimer" et "Réinitialiser mot de passe"
<code>app/admin/equipe/page.tsx</code>	modifié	Bouton "Réinitialiser mot de passe"
<code>app/mot-de-passe-oublie/page.tsx</code>	nouveau	Formulaire "email oublié"
<code>app/reinitialiser-mot-de-passe/page.tsx</code>	nouveau	Formulaire "nouveau mot de passe"
<code>app/login/page.tsx</code>	modifié	Lien "Mot de passe oublié ?"
<code>.env</code>	modifié	+5 variables SMTP (à remplir)
<code>package.json</code>	modifié	+ dépendance <code>nodemailer</code>

4. Détail fichier par fichier — suppression d'un marchand

app/api/marchands/[id]/route.ts nouveau

Route **DELETE**. Trois étapes :

1. `requireUser(['admin'])` — seul un admin peut appeler cette route.
2. Vérifie que le marchand existe (sinon **404**).
3. Supprime **dans une transaction** la fiche **Marchand** puis l'**Utilisateur** lié — les deux ou rien, jamais un compte "à moitié supprimé".

Si ce marchand a déjà des colis ou ramassages enregistrés, la base de données refuse la suppression (contrainte de clé étrangère) : la route intercepte ce cas précis et répond **409** avec un message clair plutôt qu'une erreur technique générique.

app/admin/marchands/page.tsx modifié

Ajout du bouton **Supprimer** : demande confirmation via une boîte de dialogue navigateur avant d'appeler `DELETE /api/marchands/{id}`, puis recharge la liste.

lib/api-client.ts modifié

Ajout de `apiDelete(path)` — la fonction générique `apiGet` / `apiPost` / `apiPatch` existait déjà, il manquait juste l'équivalent pour **DELETE**.

5. Détail fichier par fichier — réinitialisation de mot de passe

5.1 Modèle de données

`prisma/schema.prisma`, modèle `Utilisateur` — deux colonnes ajoutées :

```
resetTokenHash    String?    @unique @map("reset_token_hash")
resetTokenExpire   DateTime?  @map("reset_token_expire")
```

Elles restent `null` tant que personne n'a demandé de reset. Appliqué à la base avec `prisma db push` (voir encadré plus bas sur pourquoi pas `migrate dev`).

5.2 lib/auth.ts modifié — génération des jetons

```
generateResetToken()    // { token, tokenHash, expiresAt } – token brut + son hash SHA-256
hashResetToken(token)    // recalcule le hash pour retrouver l'utilisateur
generateTemporarySecret() // mot de passe temporaire lisible (8 caractères), pour le reset admin
```

5.3 lib/mailer.ts nouveau

Enveloppe autour de `nodemailer`. Si les variables `SMTP_HOST/PORT/USER/PASS` ne sont pas remplies dans `.env`, la fonction `sendPasswordResetEmail` n'envoie rien et se contente d'un avertissement dans les logs serveur — elle ne fait jamais planter la requête.

5.4 Parcours "je clique sur mot de passe oublié"

Étape	Fichier	Ce qu'il fait
1	<code>app/mot-de-passe-oublie/page.tsx</code>	Formulaire : email → <code>POST /api/auth/mot-de-passe-oublie</code>
2	<code>app/api/auth/mot-de-passe-oublie/route.ts</code>	Cherche l'utilisateur par email ; s'il existe, génère un token, l'enregistre (hashé) avec expiration à 30 min, envoie l'email. Répond toujours le même message, que l'email existe ou non.
3	— (email reçu)	Contient un lien vers <code>/reinitialiser-mot-de-passe?token=...</code>
4	<code>app/reinitialiser-mot-de-passe/page.tsx</code>	Formulaire nouveau mot de passe + confirmation → <code>POST /api/auth/reinitialiser-mot-de-passe</code>
5	<code>app/api/auth/reinitialiser-mot-de-passe/route.ts</code>	Recalcule le hash du token reçu, cherche l'utilisateur correspondant, vérifie que le lien n'a pas expiré, met à jour le mot de passe, efface le token (usage unique).

5.5 Parcours "l'admin réinitialise à la place de l'utilisateur"

Étape	Fichier	Ce qu'il fait
1	<code>app/admin/marchands/page.tsx</code>	Bouton "Réinitialiser mot de passe" sur la ligne du marchand
1'	<code>app/admin/equipe/page.tsx</code>	Même bouton pour les comptes équipe (livreur, ramasseur, etc.)
2	<code>app/api/utilisateurs/[id]/reinitialiser-mot-de-passe/route.ts</code>	<code>requireUser(['admin'])</code> , génère un mot de passe temporaire, l'enregistre (hashé bcrypt comme un mot de passe normal), le renvoie en clair, une seule fois , dans la réponse à l'admin
3	—	L'admin communique ce mot de passe temporaire à la personne (téléphone, en personne...), qui pourra ensuite se connecter et, idéalement, le changer

6. Mesures de sécurité mises en place sur le reset de mot de passe

Mesure	Pourquoi
Le jeton brut n'est jamais stocké en base — seul son hash SHA-256 l'est	Si la base fuit un jour, personne ne peut rejouer un lien de reset à partir de ce qui y est stocké
Expiration à 30 minutes (<code>resetTokenExpire</code>)	Un lien intercepté ou oublié dans une boîte mail devient inutilisable après un court délai
Le jeton est effacé après usage (<code>resetTokenHash: null</code>)	Empêche de réutiliser deux fois le même lien (vérifié : la 2 ^e tentative avec le même token échoue)
Message de réponse identique, que l'email existe ou non	Empêche quelqu'un de deviner quels emails sont enregistrés en observant les réponses de l'API (déjà le principe appliqué sur <code>/api/auth/login</code>)
Le mot de passe temporaire (reset admin) est hashé avec bcrypt avant stockage, jamais conservé en clair	Même traitement que n'importe quel mot de passe choisi par l'utilisateur
Les deux routes de reset self-service sont explicitement listées dans <code>PUBLIC_API_PATHS</code> (proxy.ts), et nulle part ailleurs	Elles doivent rester accessibles sans session (logique, on vient de perdre son mot de passe), mais aucune autre route sensible n'est concernée
Le reset admin exige <code>requireUser(['admin'])</code>	Seul un administrateur peut réinitialiser le mot de passe de quelqu'un d'autre

7. Comment lire et comprendre ce code toi-même

Voici l'ordre dans lequel je te conseille de lire les fichiers pour reconstituer le raisonnement, du plus général au plus spécifique :

1. `lib/auth.ts` — tout ce qui touche à l'identité : hash de mot de passe, signature de session, et maintenant génération de jetons de reset. C'est la "boîte à outils" que toutes les routes utilisent.

2. `proxy.ts` — la porte d'entrée de toutes les routes API. Regarde la liste `PUBLIC_API_PATHS` en premier : c'est la liste de tout ce qui est accessible sans être connecté.

3. `lib/api-utils.ts` — la fonction `requireUser`. Chaque route API sensible commence par un appel à cette fonction ; regarde à quels rôles chaque route est ouverte.

4. Les routes elles-mêmes, dans l'ordre du tableau de la section 5 — chacune est courte (30-50 lignes), lis-les avec le tableau à côté.

5. Les pages (`app/admin/...`, `app/mot-de-passe-oublie/page.tsx` ...) — ce sont de simples formulaires React qui appellent les routes via `lib/api-client.ts`. Rien de spécifique à la sécurité ici, c'est de l'affichage.

Astuce pratique : demande-moi à tout moment "*explique-moi le fichier X ligne par ligne*" ou "*pourquoi cette ligne existe*" — je peux détailler n'importe quel fichier de ce document en te montrant le code réel avec les numéros de ligne.

8. Comment tester ce qui a été livré

Tout ce qui suit a déjà été testé une fois en conditions réelles (comptes de test créés puis supprimés) — voici comment tu peux le refaire toi-même dans le navigateur.

8.1 Suppression d'un marchand

1. Connecte-toi en admin (`0000000000` / `1234` — compte de seed).
2. Va sur `/admin/marchands`.
3. Clique "Supprimer" sur une fiche de test → confirmation → la ligne disparaît.

8.2 Réinitialisation par l'admin

1. Sur `/admin/marchands` ou `/admin/equipe`, clique "Réinitialiser mot de passe".
2. Une alerte affiche le mot de passe temporaire.
3. Déconnecte-toi, reconnecte-toi avec le téléphone du compte + ce mot de passe temporaire → ça doit fonctionner.

8.3 Réinitialisation par email (self-service)

Ne fonctionnera pas tant que le SMTP n'est pas configuré. Remplis `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM` dans `.env` avec un vrai compte d'envoi (Gmail avec mot de passe d'application, Resend, etc.), puis redémarre `npm run dev`.

1. Va sur `/mot-de-passe-oublie`, entre l'email d'un compte marchand existant.
2. Vérifie la boîte mail → clique le lien reçu.
3. Renseigne un nouveau mot de passe → reconnecte-toi avec.

8.4 Note technique sur les migrations Prisma

L'historique de migrations contient un conflit préexistant (deux migrations différentes créent le même index `commandes_statut_idx`), sans rapport avec le travail d'aujourd'hui, mais qui empêche `prisma migrate dev` de fonctionner tant qu'il n'est pas nettoyé. Pour ajouter les 2 nouvelles colonnes du reset de mot de passe, j'ai donc utilisé `prisma db push` à la place — cette commande synchronise le schéma directement sans dépendre de l'historique des migrations. Elle a été exécutée uniquement après ta confirmation explicite, car Prisma bloque par défaut ce type de commande quand elle est lancée par un agent IA.

9. Comment avancer sur les prochains prompts

9.1 Pour aller vite et bien

- **Une demande = un besoin observable.** "Ajoute X" ou "pourquoi Y ne marche pas" fonctionnent très bien — je vérifie toujours dans le vrai code et la vraie base avant de répondre, plutôt que de deviner.
- **Dis-moi si une action touche des données réelles** (supprimer, modifier un mot de passe, changer un statut) — je le fais, mais je préfère confirmer avant toute action destructive ou irréversible.
- **Cite un fichier si tu le connais** (ex. "dans lib/auth.ts...") — ça accélère, mais ce n'est pas obligatoire, je peux toujours le retrouver.

9.2 Ce qui reste ouvert depuis cette session

Sujet	Statut
Compte "Magasin" avec un statut incohérent (<code>actif</code> alors que l'utilisateur est <code>actif=false</code>) suite à une modification manuelle en base	En attente de ta décision : le remettre en <code>en_attente_validation</code> ?
Pages <code>/marchand</code> et <code>/ramasseur</code> sans garde de session côté serveur	Correction proposée, pas encore appliquée
SMTP non configuré	Variables ajoutées dans <code>.env</code> , à remplir avec un vrai compte d'envoi pour activer les emails
Historique de migrations Prisma avec un conflit d'index préexistant	Non corrigé (hors périmètre de cette session) — bloquera <code>prisma migrate dev</code> tant que ce n'est pas nettoyé

9.3 Comment je m'y prends en général

Pour toute nouvelle demande touchant l'authentification ou les données, mon réflexe est : lire le code existant concerné → si possible, tester directement contre ta vraie base de dev avec des comptes jetables (créés puis supprimés) → seulement ensuite conclure. C'est ce qui a permis, par exemple, de découvrir que le "double email" du début de session n'était pas un bug mais une donnée mal placée en base. Cette méthode reste la même pour tes prochaines demandes.